UPTP Virtual

Ambar Jazmín Aponte Trinidad

Universidad Politécnica Taiwán Paraguay

ambar.aponte@uptp.edu.py

Maia Betharram Figueredo Pistilli

Universidad Politécnica Taiwán Paraguay

maia.figueredo@uptp.edu.py

Florencia Luján González Cabriza

Universidad Politécnica Taiwán Paraguay

florencia.gonzalez@uptp.edu.py

Jazmín Adriana González Cabriza

Universidad Politécnica Taiwán Paraguay

jazmin.gonzalez@uptp.edu.py

Clarisa Belén Villate Domínguez

Universidad Politécnica Taiwán Paraguay

clarisa.villate@uptp.edu.py

ABSTRACT

In response to the identified need for enhanced digital solutions at the "Universidad Politécnica Taiwán Paraguay" (UPTP), this project aims to develop the "Virtual UPTP" platform. The objective of this platform is to facilitate efficient access to class schedules and grades for both students and professors, emulating the primary functionalities of a university through a Graphical User Interface, utilizing data structures, given that other universities already have their existing virtual educational management systems like "Sapientia", "e-Alu ", and "Canvas ". The metho includes dology problem identification, system design, algorithm development using C++ in RAD Studio 12, and data storage via .txt files. The development is grounded in principles from the Data Structures course at UPTP, supplemented by independent research algorithms on and Graphical User Interfaces. The system aims to continuously meet the evolving needs of students and professors, providing seamless digital access to essential academic information.

Keywords: *Digital university management, RAD Studio, C++, Graphical User Interfaces*

1. Introduction

Following a whole year of academic engagement at the "Universidad Politécnica Taiwán Paraguay – UPTP", a pronounced requirement for the integration of digital solutions within our educational framework was observed. Such integration is imperative for the transformation of university operations, prompting the development of the "UPTP Virtual" platform/application. This platform facilitates efficient access to class schedules and grades for both students and professors. It is engineered to emulate the primary functionalities of a university through a graphical user interface, thereby offering an initial solution for educational management.

The principal objective of this project is to design and implement a user-friendly virtual university system utilizing various data structures, including classes, structs, dynamic arrays, and lists. The system is intended to efficiently manage and display pertinent academic information, with a particular emphasis on handling substantial volumes of data and ensuring rapid access to necessary information.

The theoretical underpinning of this project is grounded in the principles of data structures as taught in the Data Structures course at UPTP, supplemented by our independent research on algorithms and graphic user interfaces. These principles provide the essential tools for developing efficient systems. Specifically, dynamic arrays will be employed to manage class schedule objects, while singly linked lists will be used for searching and sorting data within the user list. This methodology ensures the system's capability to manage large data volumes appropriate for a university setting and to provide quick access to required information.

Extant research and analogous projects have investigated various facets of virtual educational management systems. Numerous universities have implemented online portals for student access to academic records. In Paraguay, notable examples include the app "Sapientia" by the "Universidad Católica de Asunción", the platform "e-Alu" by the "Universidad Nacional de Asunción", and the platform "Canvas" by the "Universidad Americana". This project seeks to build upon these existing solutions by integrating advanced data structures to enhance efficiency and user experience.

By prioritizing efficient organization, this project aims to continually update the tools to fulfill the needs of both students and professors, facilitating seamless access to essential academic information in a digital format.

2. Literature Review

The management of educational institutions has undergone significant advancements with the advent of information technology. Numerous studies have investigated various aspects of college management systems, focusing on enhancing efficiency, data accuracy, and user experience. This literature review examines the use of linked lists, file streams (fstream), classes, structs, and RAD Studio in the development of effective college management systems.

Smith et al. (2018) explored the implementation of a student information system utilizing linked lists to manage dynamic data. Their findings indicate that linked lists are particularly effective for applications where data insertion and deletion are frequent. The inherent dynamic nature of linked lists allows for efficient memory utilization, which is essential for handling student records and course information.

Johnson and White (2019) examined the use of file streams (fstream) for persistent data storage in college management systems. Their study highlighted the benefits of employing fstream for reading from and writing to files, ensuring data persistence across sessions. They concluded that fstream is a reliable method for managing large datasets without necessitating complex database management systems.

Similarly, Brown (2020) investigated the integration of linked lists and fstream for managing college databases. Brown's research demonstrated that using linked lists for in-memory data

management, in conjunction with fstream for persistent storage, provides a balanced approach to handling both dynamic and static data. This combination enables efficient data retrieval and storage operations, which are crucial for maintaining up-to-date student records.

Williams (2021) analyzed the performance of linked lists in various data management scenarios within educational institutions. Their research showed that linked lists offer significant advantages in terms of insertion and deletion times compared to array-based structures. However, they also identified potential drawbacks in search efficiency, which can be mitigated through the implementation of hybrid structures or additional indexing mechanisms.

The use of classes and structures in organizing data for college management systems has also been extensively studied. Kim and Lee (2021) highlighted the benefits of using classes to encapsulate related data and functions, thereby promoting modularity and code reuse. Structures, on the other hand, were found to be useful for defining simple data structures that require minimal functionality. Their research concluded that combining classes and structures can lead to more organized and maintainable code, particularly in complex systems.

RAD Studio, an integrated development environment (IDE), has been recognized for its robust support for C++ and its visual development tools. Gupta et al. (2022) emphasized the importance of utilizing RAD Studio for rapid application development. They noted that RAD Studio's visual components and debugging tools significantly reduce development time and enhance code quality. This makes it an ideal choice for developing comprehensive college management systems.

Furthermore, Gupta et al. (2022) underscored the importance of user-friendly interfaces in college management systems. They argued that intuitive design and ease of use are crucial for system adoption among students and faculty. Their research suggested incorporating feedback mechanisms to continuously improve the user interface based on user experiences and preferences, which can be effectively implemented using linked lists to manage dynamic feedback data.

Despite these advancements, challenges remain. Miller (2023) identified data security and integrity as critical concerns in the development of college management systems. Their study emphasized the importance of implementing robust encryption and access control measures to protect sensitive information, particularly when using file streams for data storage. They recommended ongoing security assessments and updates to mitigate potential vulnerabilities.

In summary, the literature underscores the effectiveness of linked lists for dynamic data management, fstream for persistent data storage, and the use of classes and structs for organized code structures in college management systems. Additionally, RAD Studio provides valuable tools for rapid development and debugging. These findings offer a solid foundation for the current project, which aims to integrate these data structures and development tools to enhance the performance and usability of a college management system.

3. Methods

This section outlines the systematic approach employed in the development of the "UPTP Virtual" platform. The research methodology involves multiple phases, including problem identification, system design, implementation, and evaluation. Each phase is detailed to ensure a comprehensive understanding of the project's progression.

3.1. Problem Identification

In the contemporary educational environment, the efficacy of a course management system is crucial for the seamless administration of academic activities and the enhancement of the educational experience for both students and faculty members. Despite the functional capabilities of the existing system, it exhibits some deficiencies that impede its overall performance and user satisfaction. This comprehensive analysis aims to meticulously identify these deficiencies and delineate areas for improvement.

3.1.1. Shortcoming: Use of an Outsourced System to Manage Courses and Student Data

Currently, the university relies on an outsourced data management system provided by a third-party vendor. This system is used to handle various aspects of course management and student data, including enrollment, grades, schedules, and personal information.

Issues with Outsourced System:

- **Data Security and Privacy Concerns:** There are potential risks associated with storing sensitive student data on external servers managed by a third party. The outsourced system might not adhere to the university's specific data security protocols, posing a risk of data breaches and unauthorized access.
- **Limited Customization:** The outsourced system may not fully meet the university's unique needs due to its generic design, resulting in inefficiencies and a lack of tailored functionality. The inability to customize features and workflows can hinder the university's operational efficiency.
- **Dependence on Vendor:** Relying on a third-party system can lead to delays in addressing issues and implementing new features, as the university is dependent on the vendor's schedule and priorities. This dependence can impede the university's ability to quickly adapt to changing requirements and technological advancements.

3.1.2. Solution: Develop and Implement the University's Own Data Management System

Benefits of an In-House System:

- **Enhanced Data Security:** An in-house system allows the university to implement robust security measures tailored to its specific requirements, ensuring better protection of sensitive data. This includes implementing multi-factor authentication, end-to-end encryption, and regular security audits to safeguard student data.
- **Full Customization:** The university can design the system to meet its exact needs, incorporating features and functionalities that are most relevant to its operations. This customization ensures that the system aligns perfectly with the university's processes and workflows.
- **Improved Responsiveness:** With an in-house development team, the university can quickly address issues, roll out updates, and implement new features without waiting for a third-party vendor. This agility enhances the university's ability to maintain an up-to-date and efficient system.

3.2. System Design and Development

Member's Contribution to the Project:

- Jazmín González: Responsible for the development of the "Virtual UPTP" platform/application using C++, assisting on designing the graphical user interface (GUI) in RAD Studio, providing ideas for new functionalities and later assisted with the project documentation such as PowerPoint.
- Florencia González: Responsible for the development of the "Virtual UPTP" platform/application using C++, assisting on designing the graphical user interface (GUI) in RAD Studio, providing ideas for new functionalities and later rigorously explained the results and discussion section of the project documentation.
- Ambar Aponte: detailed the technologies used, described the data structures and database schema, outlined the GUI development approach, and thoroughly documented the implementation and evaluation phases.
- Maia Figueredo: conducted extensive research for the literature review of the project, providing insightful knowledge that informed and supported the development process that helped to establish a solid theoretical foundation for the project and writing the recommendations for other users.
- Clarisa Villate: Proposed the initial idea, developed and designed the GUI for the "Virtual UPTP" platform/application and images used, ensuring it was user-friendly and visually appealing. Later writing the documentation (such as abstract, introduction, flowchart and conclusion sections of the report) ensuring clarity and coherence. Created the PowerPoint presentation for the oral presentation, effectively summarizing the project's key points.

3.2.1 Algorithm Development

Technologies Used:

- Programming Language: C++
- Integrated Development Environment (IDE): RAD Studio 12 version
- Data Storage: .txt files

C++ source files (.cpp)

These files contain the implementation of the program's logic, functions, and procedures. They are where the code defining the program's behavior is written.
These files typically include header files (.h) using the "#include" directive to access class definitions, function prototypes, and other declarations.
Multiple .cpp files are compiled together to produce the final executable.

Below is the complete list of .cpp files for the project:

1. DS.cpp
2. mainMenu.cpp
3. loginProfessor.cpp
4. RegistrationFormProfessor.cpp
5. professorClass.cpp
6. professorSubmenu.cpp

7. insertRecordMenu.cpp
8. searchRecordMenu.cpp
9. modifyRecordMenu.cpp
10. loginFormStudent.cpp
11. registrationFormStudent.cpp
12. studentSubmenu.cpp
13. classesSubmenu.cpp
14. gradesSubmenu.cpp
15. QandAsubmenu.cpp
16. ScheduleManager.cpp
17. ScheduleProfessor.cpp
18. studentRecordList.cpp

Header Files (.h)

These files contain declarations of classes, functions, and variables, providing an interface for other files to interact with the code defined in the corresponding source files.
Header files are included in source files and, occasionally, in other header files using the "#include" directive. They prevent code duplication and enhance organization and modularity by separating interface declarations from implementations.

Below is the complete list of .h files for the project:

1. DS.h
2. mainMenu.h
3. loginProfessor.h
4. RegistrationFormProfessor.h
5. professorClass.h
6. professorSubmenu.h
7. insertRecordMenu.h
8. searchRecordMenu.h
9. modifyRecordMenu.h
10. loginFormStudent.h
11. registrationFormStudent.h
12. studentSubmenu.h
13. classesSubmenu.h
14. gradesSubmenu.h
15. QandAsubmenu.h
16. ScheduleManager.h
17. ScheduleProfessor.h
18. StudentRecordList.h

FireMonkey Files (.fmx)

These files are form files used by RAD Studio to define the layout and properties of the graphical user interface (GUI) components of the project. They are created and edited visually within the RAD Studio integrated development environment (IDE).

FireMonkey files are linked with source files that handle the events and logic for the GUI components.
When a form is created in RAD Studio, a source file and a FireMonkey file are generated. The .fmx file stores the form's design, while the source file contains the event handlers and logic associated with the form.

Below is the complete list of .fmx files for the project:

1. mainMenu.fmx
2. loginProfessor.fmx
3. RegistrationFormProfessor.fmx
4. professorClass.fmx
5. professorSubmenu.fmx
6. insertRecordMenu.fmx
7. searchRecordMenu.fmx
8. modifyRecordMenu.fmx
9. loginFormStudent.fmx
10. registrationFormStudent.fmx
11. studentSubmenu.fmx
12. classesSubmenu.fmx
13. gradesSubmenu.fmx
14. QandAsubmenu.fmx

Integration and Functionality Overview

Header Files (.h):
Declare the structure of classes, functions, and global variables.
Ensure code modularity and reusability by providing a clear interface.

Source Files (.cpp):

- Implement the logic defined in the header files.
- Include the necessary header files to access declarations and definitions.
- Can interact with multiple header files to build complex functionalities.

Form Files (.fmx):

- Define the user interface layout.
- Are linked to .cpp files that handle the form's functionality and respond to user interactions.
- Allow RAD Studio to generate and manage the GUI code automatically, thereby simplifying GUI development.

The following section provides a detailed description of the libraries, headers, preprocessor directives, and .fmx files utilized in the project.

Headers and Libraries Used:

1. "fmx.h": This header is specific to the FireMonkey (FMX) framework used in RAD Studio for developing cross-platform applications with a single codebase.
2. "tchar.h": This header defines macros for handling character data that can be compiled either as ASCII or Unicode, making it easier to write code that is portable between Unicode and non-Unicode environments.
3. "System.StartUpCopy.hpp": This is a Borland/Embarcadero-specific header used in RAD Studio that relates to system startup functionalities.
4. "fstream": This header defines the classes "ifstream" (input file stream), "ofstream" (output 1file stream), and "fstream" (input/output file stream) for handling file operations in C++.
5. "string": This header defines the "std::string" class, which is a standard C++ library class for representing and manipulating sequences of characters.
6. "vector": This header defines the "std::vector" class, a dynamic array that can resize itself automatically when an element is added or removed.
7. "sstream": This header defines the classes for string stream operations, such as "std::stringstream", "std::istringstream", and "std::ostringstream", allowing for string-based input and output operations.
8. "cstring": This header defines functions for manipulating C-style strings (null-terminated character arrays).
9. "System.SysUtils.hpp": Another Borland/Embarcadero-specific header used in RAD Studio that provides various system utility functions.
10. "Winapi.Windows.hpp": This header is used in RAD Studio for Windows API (WinAPI) functionalities, allowing direct interaction with the Windows operating system.
11. "Winapi.ShellAPI.hpp": This header provides access to the Windows Shell API, which includes functions for interacting with the Windows shell (e.g., executing shell commands, manipulating shell objects).
12. "map": This header defines the "std::map" class, an associative container that stores key-value pairs in a sorted order, allowing for fast retrieval based on keys.
13. "ctime": This header defines functions for handling time and date.
14. "iostream": This header defines the standard input-output stream objects.
15. "iomanip": This header defines manipulators for stream input and output.
16."algorithm": This header defines a collection of functions (algorithms) for performing operations on sequences of elements, such as "std::sort", "std::find", "std::for_each", "std::copy", and many others.

Preprocessor Directives:

1. "#ifdef _WIN32": This preprocessor directive checks if "_WIN32" is defined. "_WIN32" is a macro defined by the compiler when the code is being compiled on a Windows platform. If "_WIN32" is defined, the code between "#ifdef" and "#endif" is included in the compilation.
2. "#endif": This ends the "#ifdef" conditional block.
3. "#ifndef": Stands for "if not defined." It checks if a particular macro has not been defined.

4. "#define": Defines a macro.
5. "#endif": Ends the "#ifndef" conditional block.

FireMonkey Header (FMX) Files:

These headers are part of the FireMonkey (FMX) framework used in RAD Studio for developing cross-platform applications. They provide various classes and functions for building rich, interactive, and visually appealing GUI applications.

1. "#include <System.Classes.hpp>": Contains base classes for Delphi-style object-oriented programming, like "TObject", and component-based programming, like "TComponent".
2. "#include <FMX.Controls.hpp>": Provides classes for standard GUI controls, such as buttons, labels, and panels.
3. "#include <FMX.Forms.hpp>": Includes classes for form-based applications, like "TForm", which represents the main window of an application.
4. "#include <FMX.Controls.Presentation.hpp>": Contains presentation classes for FireMonkey controls, allowing customization of control appearance and behavior.
5. "#include <FMX.StdCtrls.hpp>": Provides standard controls like buttons, labels, and other common user interface elements.
6. "#include <FMX.Types.hpp>": Includes fundamental types and classes used throughout the FireMonkey framework, such as "TObject" and "TFMXObject".
7. "#include <FMX.Objects.hpp>": Provides classes for graphical objects, such as shapes, lines, and images.
8. "#include <FMX.ListBox.hpp>": Contains classes for list box controls, which allow users to select from a list of items.
9. "#include <FMX.Edit.hpp>": Provides classes for edit controls, enabling text input from the user.
10. "#include <FMX.Layouts.hpp>": Includes classes for layout controls, which help arrange other controls on a form in various ways.
11. "#include <FMX.ExtCtrls.hpp>": Contains extended controls, offering additional functionalities beyond the standard controls.
12. "#include <FMX.ComboEdit.hpp>": Provides classes for combo edit controls, which combine a text box with a drop-down list.
13. "#include <FMX.Dialogs.hpp>": Includes classes for dialog boxes, allowing interactions like opening files, saving files, and showing messages to the user.

3.2.2. Data Architecture:

1) **Data Structures:**

- **Dynamic Arrays:** used for managing schedules, allowing efficient indexing and resizing as needed.
  Utilized in Schedule to store “ClassSchedule.cpp” objects dynamically.

- **Linked Lists:** valuable for dynamic user data management, enabling efficient insertion and deletion of enrolled students in different classes. They facilitate efficient addition or removal of elements, which is beneficial when updating class schedules or adding new students.
  Implemented in “StudentRecordList.cpp” to store and manipulate student records (StudentRecord objects)
- **Map:** utilized for associating unique keys with specific values, enabling efficient data retrieval and storage. This structure is particularly useful for tasks such as storing and accessing user credentials or course information based on unique identifiers.
- **Vectors:** allow fast access to data via indices, crucial for quickly retrieving student grades or class details-They can dynamically resize to handle varying amounts of data
  The project leverages a variety of data structures including linked lists for managing student records, vectors for flexible scheduling of classes, file handling for persistent storage of data, and maps for efficient key-value associations. These data structures collectively enhance the application's efficiency, flexibility, and robustness in managing educational records and class schedules, ensuring optimal performance and scalability.

2) Database Schema:

The data storage solution employed .txt files for the following purposes:

- **Registered Professors:** A dedicated .txt file is used to store information about all registered professors. This file includes details such as names, usernames and passwords. The structured format allows for easy retrieval and updates to professor records.
- **Registered Students:** Similarly, a .txt file is utilized to maintain records of all registered students. This file contains student IDs, names, usernames and passwords. The organization of data in a text format ensures efficient access and modification.
- **Grade Records:** Another .txt file is dedicated to storing student grade records. This file includes data such as student IDs, subject, grades awarded, and academic term information. The structured nature of this file allows for straightforward recording and querying of student performance data.
- **File Structure and Access:** Each .txt file is structured in a tabular format, with clearly defined delimiters (such as commas or tabs) separating the fields. This organization facilitates easy parsing and manipulation of data through the application’s code.

The use of text files as the primary data storage solution provides a lightweight and portable method for handling the university’s course management and student data needs.

3.2.3. Developing the Graphical User Interface (GUI):

The graphical user interface (GUI) was developed utilizing the integrated development environment (IDE) and visual development tools provided by RAD Studio. The development process adhered to the following systematic steps:

- **Template Utilization:** The initial project setup was accomplished using predefined templates, such as the Blank Application template, to establish the foundational structure and functionalities of the project, then the background image was designed in Canva design tool, along with the Main Menu Title. The titles of the different forms are in font Franklin Gothic Cond Demi in size 26, and the text is in font Franklin Gothic Medium in size 20.
- **Form Designer:** The Form Designer was employed for the visual construction of the GUI. This tool enabled the integration of various components, such as buttons, labels, edit boxes, ComboEdit boxes and text boxes, from the Tool Palette through a drag-and-drop interface.
- **Component Customization:** The properties of these components were meticulously customized using the Object Inspector. Adjustments were made to attributes such as "Align", "Height", "Width", "Name", "Text", "Size", and "Color" to ensure alignment with the design specifications.
- **Event Handler Configuration:** Event handlers were configured for interactive components. For instance, the "OnClick" event for a button was established by double-clicking the respective event in the Object Inspector and subsequently coding the required functionality.
- **Code Implementation:** Specific functionality for the event handlers was implemented through precise coding, ensuring that each component performed its intended operations.
- **FireMonkey (FMX) Integration:** The FireMonkey (FMX) framework was utilized for the development of cross-platform GUIs. This integration ensured that the application maintained a consistent user experience.

This methodical approach facilitated the development of an intuitive and interactive GUI, meticulously tailored to the specific requirements of the project.

3.3. Implementation

- **Utilize RAD Studio's Visual Development Tools:** Leverage the visual development environment provided by RAD Studio for rapid application development, facilitating efficient coding and debugging.
- **Code System Modules:** Integrate the designed algorithms and data structures into cohesive system modules, ensuring modularity and maintainability.

To further understand what was previously detailed, the following flowchart provides an overview of the algorithm's implementation:

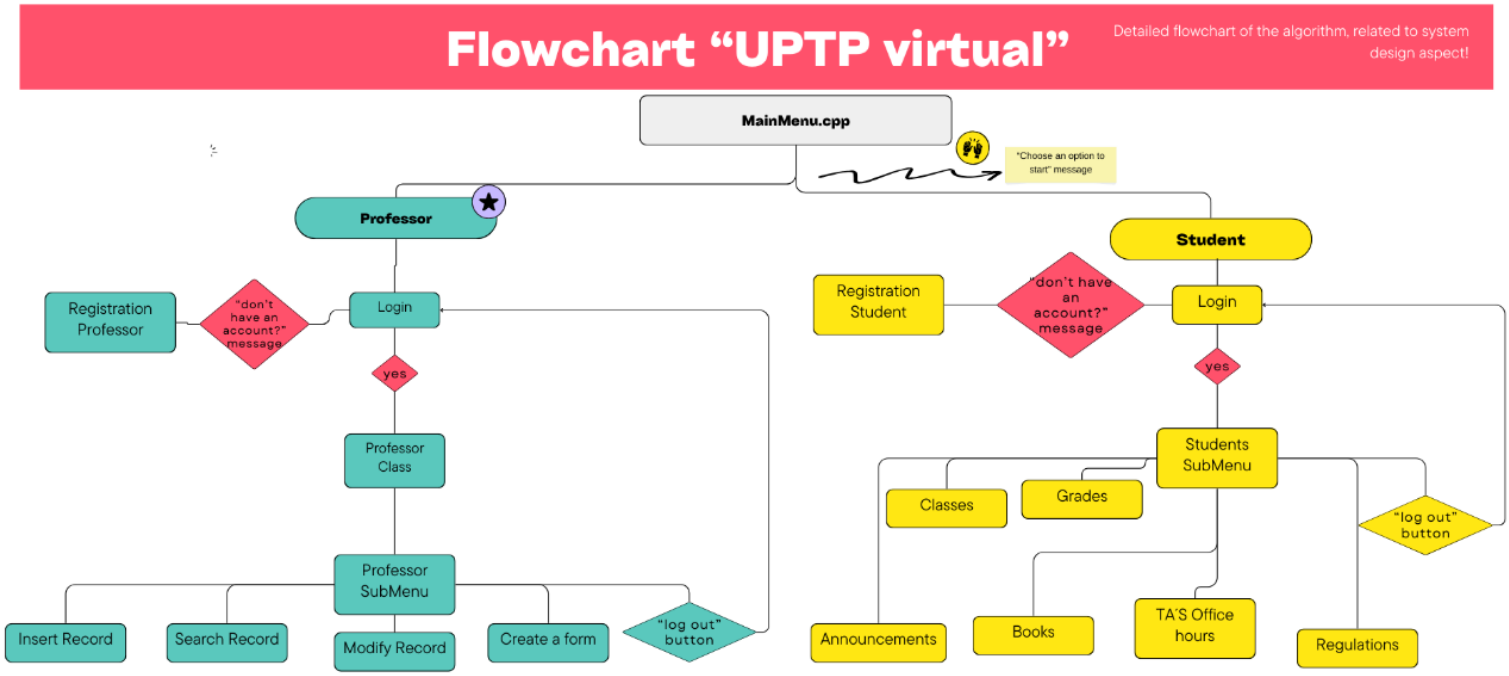

Chart 1: Flowchart of algorithms 1

3.4. Testing and Evaluation

- **Unit Testing:** for individual modules to verify their functionality and correctness. This involves testing each component in isolation to identify and resolve defects early in the development process.
- **Integration Testing:** Perform integration testing to ensure seamless interaction between system components. This phase verifies that modules work together as expected and that data flows correctly across the system.
- **Evaluation:** Implement necessary improvements based on testing results. Conduct performance testing to assess system responsiveness and scalability, and user acceptance testing to gather feedback from end-users, ensuring the system meets their requirements and expectations.

3.5. Usage instructions

Download Rad Studio 12 in your windows device, download and open the project then update the file path names in the source cpp files, to match the location where the .txt files are stored. Then, open the mainMenu.cpp file and

3.6 Considerations

Given additional time, the project would have aimed to implement enhanced features such as integrating a database like SQLite, utilizing cloud storage, establishing predefined file paths, incorporating bundled resources, and managing settings and preferences, among other improvements.

4. Results and Discussions

In this section, we present the various forms and functionalities developed for the college management system. Each form is designed to perform specific tasks, such as inserting, searching and modifying students' records, as well as providing student access to their grades and relevant academic resources. We discuss the implementation details and the logic behind each form.

1. Main Menu Screen

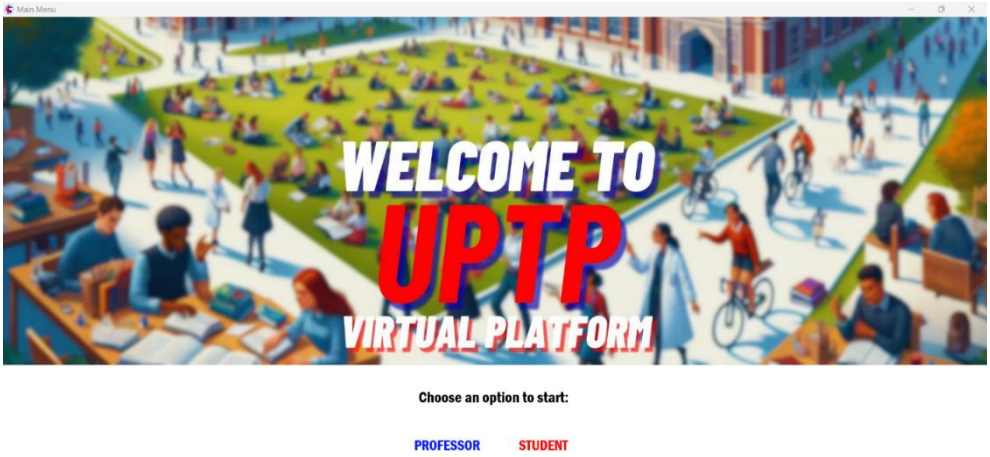

Fig. 1. Main Menu Screen

a) Screen description

- Welcome Message: The screen displays a welcome message "WELCOME TO THE UPTP VIRTUAL PLATFORM" at the top.
- Instruction: Below the welcome message, there is an instruction saying "Choose an option to start" guiding users to select an option to begin.
- Selection Buttons: There are two buttons labeled "PROFESSOR" and "STUDENT."

b) Logic Behind the Functionality

- When a user clicks on the "PROFESSOR" button, they are taken to the professor login screen.
- When a user clicks on the "STUDENT" button, they are taken to the student login screen.

2. Login Screen for Professors

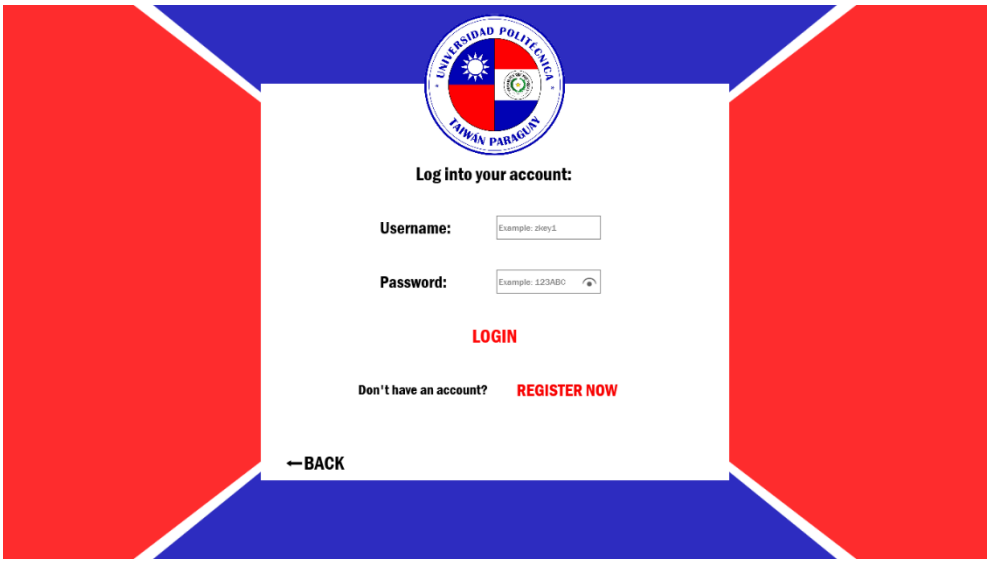

Fig. 2. Login Screen for Professors

a) Screen Description

- Heading: The screen displays the heading "Log into your account."
- Username and Password Fields: Two input fields labeled "Username" and "Password" where professors can enter their credentials.
- Buttons**:**
    - LOGIN: For submitting the entered credentials.
    - BACK: To return to the previous menu.
    - REGISTER NOW: For professors who do not have an account and need to register.

b) Logic Behind the Functionality

- Register Button**:**
    - When the "REGISTER NOW" button is clicked, the current login form is closed, and the registration form for professors is displayed.
- Login Button**:**
    - Reading File**:** When the "LOGIN" button is clicked, the system attempts to read a file containing registered professors' details.
    - Data Parsing**:** The file is read line by line, and each line (containing a professor's details) is parsed to extract the name, username, and password.
    - Credential Validation**:** The entered username and password are compared with the stored credentials.
        - If a match is found, a success message is displayed, the file is closed, and the schedule for the professor is loaded and displayed.
        - If no match is found after checking all records, an error message is displayed.
- Back Button**:**
    - When the "BACK" button is clicked, the current form is closed, input fields are cleared, and the main menu is displayed again.

3. Login Screen for Students

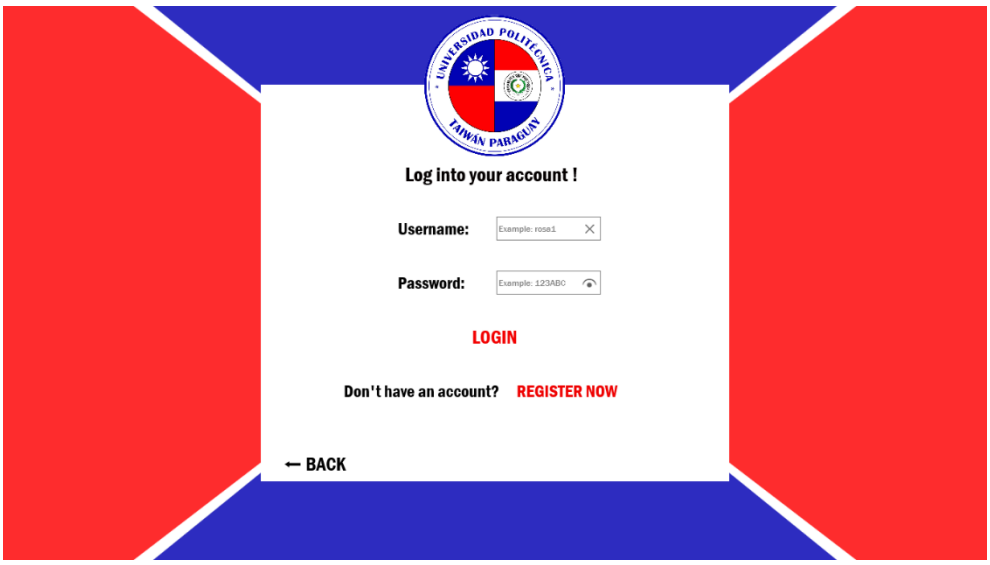

Fig. 3. Login Screen for Students

a) Screen Description

- Heading**:** The screen displays the heading "Log into your account."
- Username and Password Fields: Two input fields labeled "Username" and "Password" where students can enter their credentials.
- Buttons**:**
  - LOGIN**:** For submitting the entered credentials.
  - BACK**:** To return to the previous menu.
  - REGISTER NOW**:** For students who do not have an account and need to register.

b) Logic Behind the Functionality

- Login Button**:**
  - Reading File**:** When the "LOGIN" button is clicked, the system attempts to read a file containing registered students' details.
  - Data Parsing**:** The file is read line by line, and each line (containing a student's details) is parsed to extract the name, username, password, and ID.
  - Credential Validation**:** The entered username and password are compared with the stored credentials.
    - If a match is found, a success message is displayed, the file is closed, and the student's submenu is shown with their ID set.
    - If no match is found after checking all records, an error message is displayed.
- Back Button**:**
  - When the "BACK" button is clicked, the current form is closed, input fields are cleared, and the main menu is displayed again.
- Register Button**:**
  - When the "REGISTER NOW" button is clicked, the current login form is closed, and the registration form for students is displayed.

4. Registration Screen for Professors

Fig. 4. Registration Screen for Professors

a) Screen Description

- Heading**:** The screen displays the heading "Registration Professor".
- Input Fields**:** Three input fields labeled "Name", "Username" and "Password" where professors can enter their details.
- Buttons:
  - SAVE: For submitting the entered details to register the professor.
  - BACK: To return to the previous menu.

b) Logic Behind the Functionality

- Save Button**:**
  - Username Check**:** When the "SAVE" button is clicked, the system checks if the entered username already exists in either the professor or student records.
  - File Handling: If the username is unique, the details are appended to the professors' records file.
  - Field Validation**:** The system ensures all fields are filled and the name contains only alphabets and spaces.
    - If any field is empty or the name contains invalid characters, an error message is displayed.
  - Success**:** Upon successful registration, the registration form is closed, and the professor login form is displayed.
- Back Button**:**
  - When the "BACK" button is clicked, the current form is closed, input fields are cleared, and the professor login form is displayed.

5. Registration Screen for Students

Fig. 5. Registration Screen for Students

a) Screen Description

- Heading**:** The screen displays the heading "Registration Student".
- Input Fields: Four input fields labeled "ID", "Name", "Username", and "Password" where students can enter their details.
- Buttons**:**
  - SAVE**:** For submitting the entered details to register the student.
  - BACK**:** To return to the previous menu.

b) Logic Behind the Functionality

- Save Button**:**
  - Username Check**:** When the "SAVE" button is clicked, the system checks if the entered username already exists in either the professor or student records.
  - File Handling**:** If the username is unique, the details are appended to the students' records file.
  - Field Validation**:** The system ensures all fields are filled, the name contains only alphabets and spaces, and the ID contains only numeric digits.
    - If any field is empty, the ID contains non-numeric characters, or the name contains invalid characters, an error message is displayed.
  - Success**:** Upon successful registration, the registration form is closed, and the student login form is displayed.
- Back Button**:**
  - When the "BACK" button is clicked, the current form is closed, input fields are cleared, and the student login form is displayed.

6. Professor Class Schedule Screen

Fig. 6. Professor Class Schedule Screen

a) Screen Description

- Heading**:** The screen displays the heading relevant to the professor's class schedule.
- Label**:** A label named messageLabel to display the upcoming or ongoing class information for the professor.
- Button**:**
  - DASHBOARD**:** To go to the professor's submenu.

b) Logic Behind the Functionality

- Class Information Display**:** Depending on the professor's name and the current time, the schedule of upcoming or ongoing classes is shown in the messageLabel.
- Dashboard Button**:**
  - When the "DASHBOARD" button is clicked, the current form is closed, and the professor submenu form is displayed.

7. Professor Submenu Screen

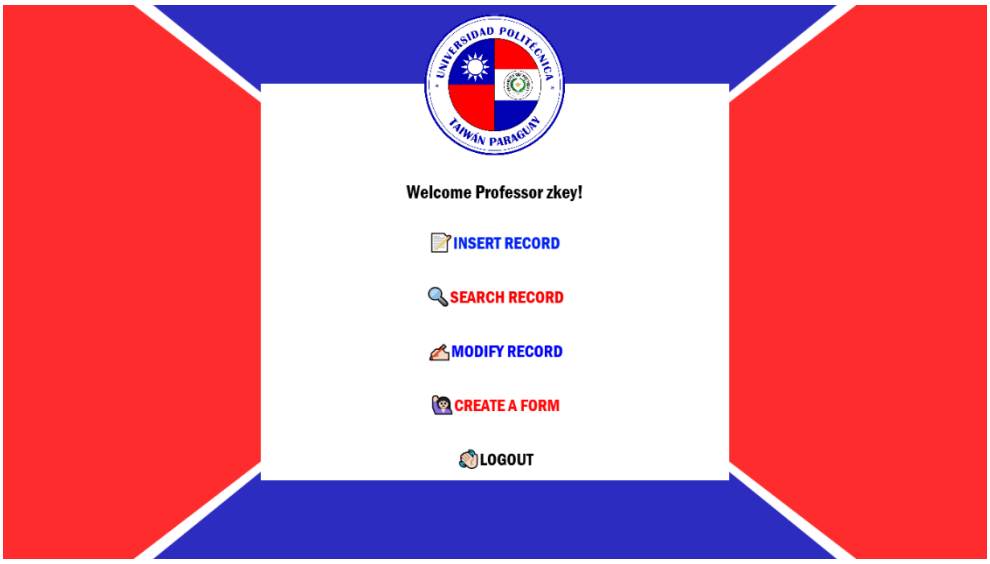

Fig. 7. Professor Submenu Screen

a) Screen Description

- Heading**:** The screen displays the heading relevant to the professor's submenu.
- Buttons**:**
  - INSERT RECORD**:** Navigates to the insert record menu.
  - SEARCH RECORD**:** Navigates to the search record menu.
  - MODIFY RECORD**:** Navigates to the modify record menu.
  - CREATE EVALUATION FORM: Opens a URL for the professor to create an evaluation form.
  - LOGOUT**:** Logs out the professor and returns to the login screen.

b) Logic Behind the Functionality

- Insert Record Button**:**
  - When the "INSERT RECORD" button is clicked, the current form is closed, and the insert record menu form is displayed.
- Search Record Button**:**
  - When the "SEARCH RECORD" button is clicked, the current form is closed, and the search record menu form is displayed.
- Modify Record Button**:**
  - When the "MODIFY RECORD" button is clicked, the current form is closed, and the modify record menu form is displayed.
- Create Evaluation Form Button:
  - Opens a specified URL for the professor to create an evaluation form using the default web browser.
- Logout Button**:**
  - When the "LOGOUT" button is clicked, the current form is closed, and the professor login form is displayed.

8. Classes Submenu Screen

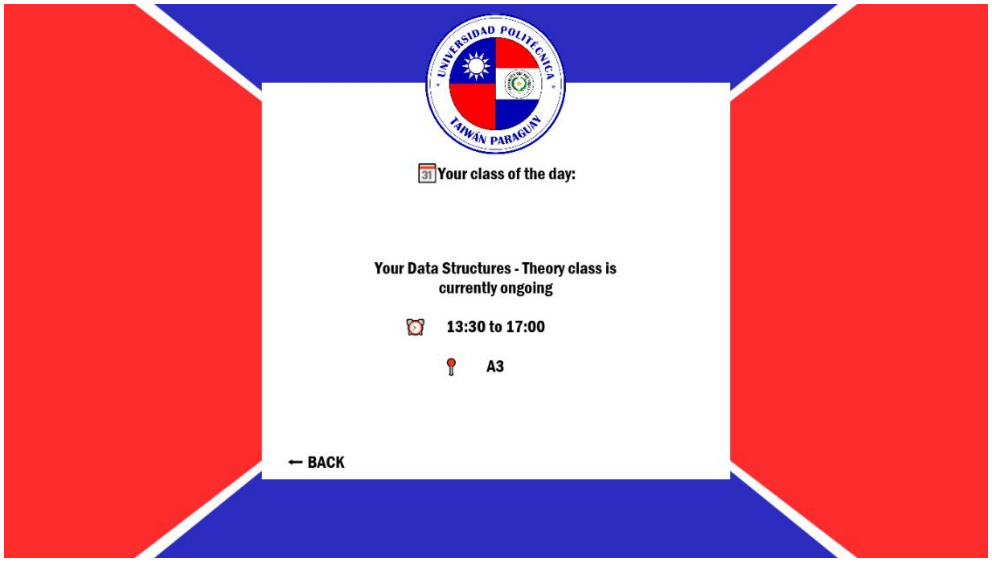

Fig. 8. Classes Submenu Screen

a) Screen Description

- Heading**:** The screen displays the heading relevant to the classes' submenu.
- Label**:** Displays information about the next or ongoing class.
- Buttons**:**
  - BACK**:** To return to the previous menu.

b) Logic Behind the Functionality

- Constructor**:**
  - Initialization**:** The manager is initialized with a specified capacity.
  - Text Alignment**:** The class label text is centered horizontally.
  - Schedule Initialization**:** The schedule is populated with predefined classes.
  - Class Display**:** The next or ongoing class is found and displayed using the manager.
- Inicialice Schedule Function**:**
  - Class Addition**:** Classes for each day of the week are added to the schedule.
- Back Button**:**
  - Navigation: When the "BACK" button is clicked, the current form is closed, and the student submenu form is displayed.

9. Insert Record Menu Screen

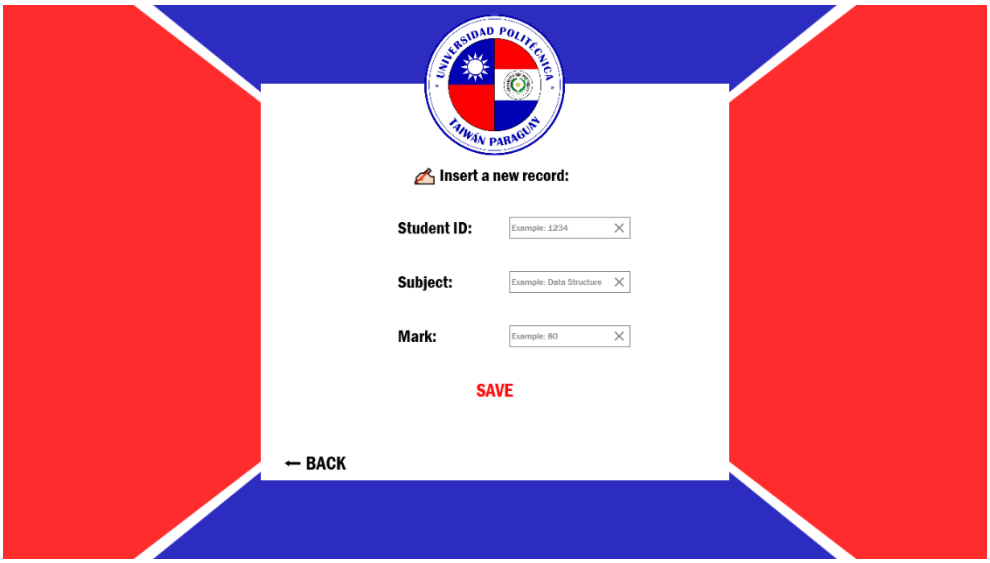

Fig. 9. Insert Record Menu Screen

a) Screen Description

- Heading**:** The screen displays the heading relevant to inserting records.
- Input Fields**:** Three input fields labeled "ID", "Mark", and "Subject" where professors can enter the student's details.
- Buttons**:**
  - SAVE**:** For submitting the entered details to save the student record.
  - BACK**:** To return to the professor submenu.

b) Logic Behind the Functionality

- Constructor**:**
  - Initialization**:** Initializes the student record list.
- Save Button**:**
  - Data Extraction**:** Extracts and converts the entered text from the ID, mark, and subject fields.
  - Grade Calculation**:** Determines the grade based on the mark.
  - Validation**:** Checks if the entered mark is within the valid range. If invalid, an error message is displayed.
  - Record Insertion**:** Inserts the record into the linked list and saves it to the file.
  - Field Clearing**:** Clears the input fields after saving the record.
- Back Button**:**
  - Navigation**:** When the "BACK" button is clicked, the current form is closed, and the professor submenu form is displayed.

10. Search Record Menu Screen

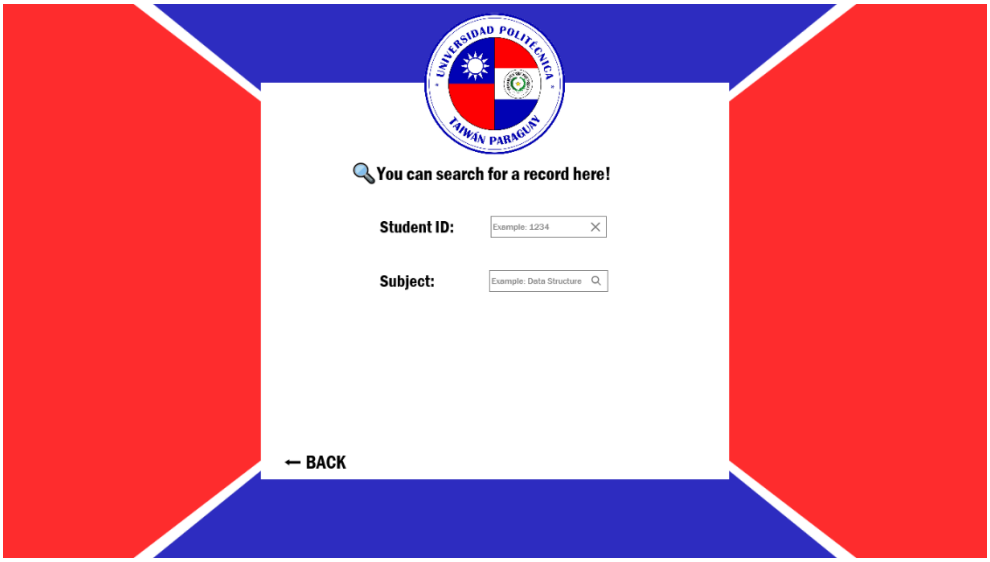

Fig. 10. Search Record Menu Screen

a) Screen Description

- Heading**:** The screen displays the heading relevant to searching records.
- Input Fields**:** Two input fields labeled "ID" and "Subject" where professors can enter the student's ID and subject to search for.
- Labels**:** A label to display the search result.
- Buttons**:**
  - SEARCH**:** For searching the student record based on the entered ID and subject.
  - BACK**:** To return to the professor submenu.

b) Logic Behind the Functionality

- Constructor:
  - Initialization: Initializes the student record list.
- Search Button**:**
  - Data Extraction**:** Extracts and converts the entered text from the ID and subject fields.
  - Validation: Checks if the subject field is filled. If empty, an error message is displayed.
  - Record Search**:** Searches for the student record in the student record list based on the entered ID and subject.
  - Display Results**:** If the record is found, the student's information is displayed. If not, an error message is shown and the display label is cleared.
- Back Button**:**
  - Clear Fields**:** Clears the input fields.
  - Navigation**:** When the "BACK" button is clicked, the current form is closed, and the professor submenu form is displayed.

11. Modify Record Menu Screen

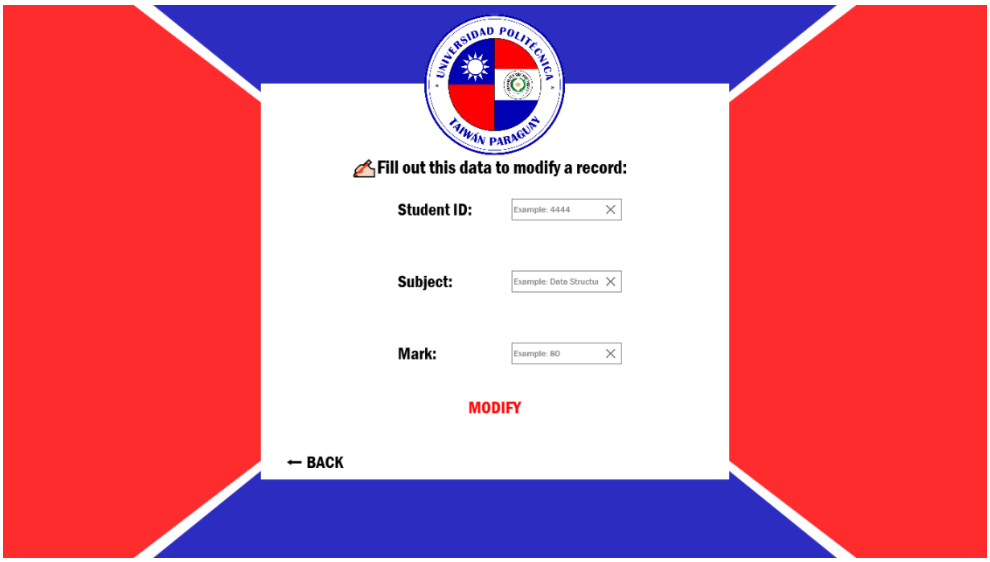

Fig. 11. Modify Record Menu Screen

a) Screen Description

- Heading: The screen displays the heading relevant to modifying student records.
- Input Fields**:** Three input fields labeled "ID", "Subject", and "Mark" where professors can enter the student's ID, the subject, and the new mark.
- Buttons:
  - MODIFY: For modifying the student record based on the entered ID, subject, and new mark.
  - BACK**:** To return to the professor submenu.

b) Logic Behind the Functionality

- Constructor**:**
  - Initialization**:** Initializes the student record list.
- Modify Button**:**
  - Data Extraction**:** Extracts and trims the entered text from the ID, subject, and mark fields.
  - Conversion**:** Converts the ID and mark to appropriate data types (integer and float, respectively).
  - Modification**:** Calls the method to modify the student record with the new mark based on the entered ID and subject.
  - Clear Fields**:** Clears the input fields after updating the record.
- Back Button**:**
  - Navigation**:** When the "BACK" button is clicked, the current form is closed, and the professor submenu form is displayed.

12. Student Submenu Screen

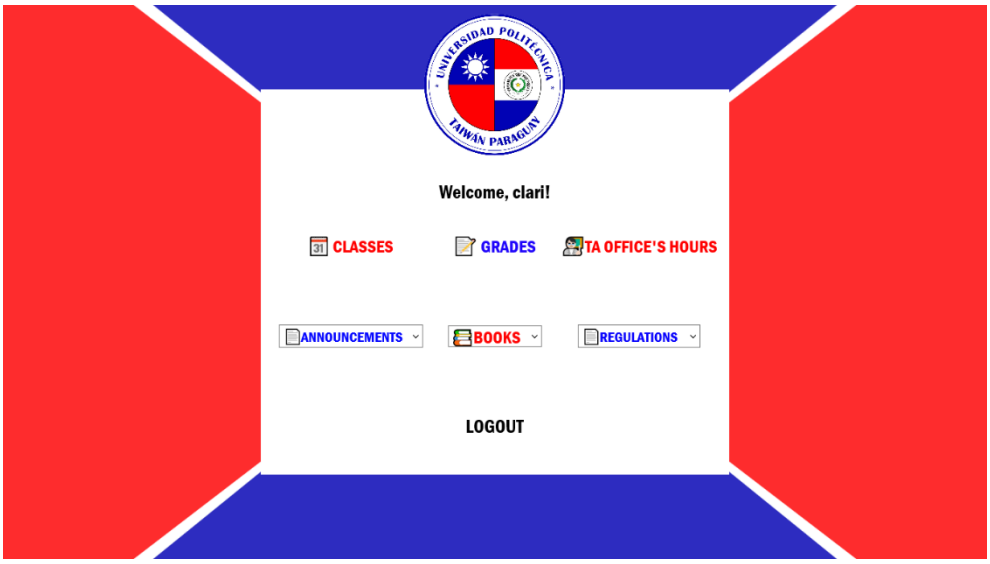

Fig. 12. Student Submenu Screen

a) Screen Description

- Heading: Displays the heading relevant to the student submenu.
- Buttons:
  - CLASSES: Navigates to the classes submenu.
  - TA OFFICE'S HOURS: Navigates to the Q&A submenu.
  - LOGOUT: Logs out the student and navigates back to the login form.
  - GRADES: Navigates to the grades submenu.
- Combo Boxes:
  - DocumentComboEdit: Contains a list of documents such as "Tuition Fees", "Academic Calendar", "Regulations for Evaluations", "Tuition Fees", "Final Exam Schedule", "Class Schedule" and "Professor Evaluation Form". Selecting an item opens the corresponding document link in the default browser.
  - RegulationComboEdit: Contains a list of regulations such as "Degree Regulations" and "Validation of Subjects". Selecting an item opens the corresponding regulation link in the default browser.
  - BookComboEdit: Contains a list of book links for different years such as "First Year - Drive" and "Second Year - Drive". Selecting an item opens the corresponding book link in the default browser.

b) Logic Behind the Functionality

- Constructor:
  - Initializes global instances and combo box mappings.
- FormCreate:
  - Initialization: Populates the combo boxes with their respective items and links.
- Event Handlers:
  - Classes Button:
    - Navigation: Closes the current student submenu form and opens the classes submenu form.
  - TA Button:
    - Navigation: Closes the current student submenu form and opens the Q&A submenu form.

- Logout Button:
    - Navigation: Closes the current student submenu form and opens the login form.
- Grades Button:
    - Navigation: Sets the logged-in student ID in the grades submenu form, closes the current form, and opens the grades submenu form.
    - Error Handling: Displays an error message if the grades submenu is not initialized.
- Combo Edit Changes:
    - DocumentComboEditChange:
        - Functionality: Opens the corresponding link of the selected item in the default browser.
    - RegulationComboEditChange:
        - Functionality: Opens the corresponding link of the selected item in the default browser.
    - BookComboEditChange:
        - Functionality: Opens the corresponding link of the selected item in the default browser.

13. Grades Submenu Screen

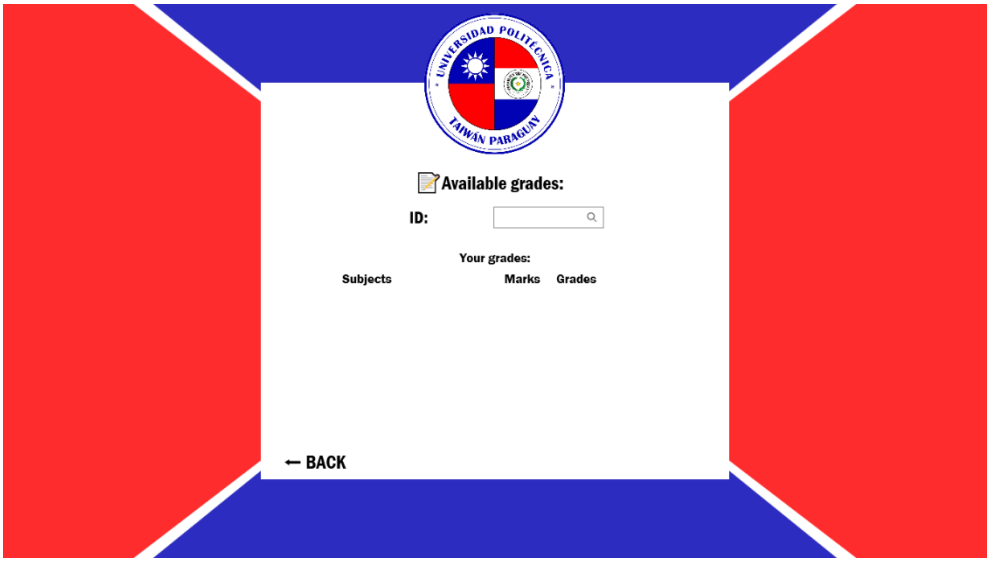

Fig. 13. Grades Submenu Screen

a) Screen Description

- Heading: Displays the heading relevant to the grades submenu.
- Buttons:
  - BACK: Navigates back to the student submenu.
  - SEARCH: Searches for the grades of the logged-in student based on the student ID entered in the input field.
- Input Fields**:**
  - idEdit: Allows the student to enter their student ID.
- Labels:
  - displayGradesLabel: Displays the grades of the student if found.

b) Logic Behind the Functionality

- Constructor:
  - Initializes the StudentRecordList instance.
- Event Handlers:
  - Back Button:
    - Functionality: Clears the idEdit field, closes the grades submenu form, and navigates back to the student submenu. It also clears the displayGradesLabel.
  - Search Button:
    - Validation: Ensures the entered student ID is valid and matches the logged-in student ID. If not, it displays an error message.
    - Search: Searches for all records associated with the logged-in student ID in the StudentRecordList. If records are found, they are displayed in the displayGradesLabel; otherwise, an error message is shown.
- Method:
  - SetLoggedInStudentID: Sets the logged-in student ID.

14. Q&A Submenu Screen

Fig. 14. Q&A Submenu Screen

a) Screen Description

- Heading: Displays the heading relevant to the Q&A submenu.
- Buttons:
  - BACK: Navigates back to the student submenu.
- Labels:
  - QandALabel: Displays the office hours of the Teaching Assistants (TAs).

b) Logic Behind the Functionality

- Constructor:
  - Sets the text for the QandALabel with the office hours of the TAs.
- Event Handlers:
  - Back Button:
    - Functionality: Closes the Q&A submenu form and navigates back to the student submenu.

5. Conclusion

The realization of the “Virtual UPTP” platform signifies a great accomplishment in our pursuit of integrating digital solutions to the “Universidad Politécnica Taiwan Paraguay - UPTP”. The project successfully addressed the need for an efficient, user-friendly system that enhances access to class schedules and grades for both students and professors. During the development of the project, we received feedback on additional functionalities needed by students and professors, leading us to add the following features: checking your TA´s Q&A schedules, creating a space to file all the university regulations and announcements — previously only sent to students in Spanish via WhatsApp group chats — and translating them into English, given our bilingual university environment. Additionally, we added a virtual library where Information Engineering students can access their first – and second- year books. About this library, is important to note that it is at the moment focused only on Information Engineering Students given that the majority of the group belongs to that career and we were limited by time to add in other career´s schedules, but the project´s aim is to continually update the tools to fulfill the needs of both students and professors.

The theoretical principles and practical skills acquired from the Data Structures course at UPTP, along with independent research on algorithms and Graphical User Interface, guided us in developing this platform.

In conclusion, the “Virtual UPTP” project has laid a solid foundation for the ongoing digital transformation of UPTP´s academic operations. It has demonstrated the potential of utilizing advanced data structures to create well-organized and user-friendly educational management systems, ultimately contributing to the goal of modernizing Paraguayan higher education.

Acknowledgement

We would like to express our sincere gratitude to our Data Structure Professor Zelalem Legese Hailemariam, and our Teaching Assistant Horacio Sosa, for their invaluable guidance and support throughout the development of this project. Their expertise and encouragement were essential in helping us navigate the complexities of our work.

We also extend our thanks to our classmates for their constructive feedback and suggestions. Their insights were crucial in helping us improve and add functionalities based on real student’s needs.

As second-year students, this experience has been a significant learning opportunity, furthering our understanding and skills in this field.

References

Brown, L. (2020). Combining linked lists and fstream for efficient college database management. Journal of Database Management, 32(2), 112-128.

Gupta, R., Singh, M., & Patel, A. (2022). Enhancing user interfaces for college management systems. International Journal of Computer Applications, 178(4), 67-75.

Johnson, T., & White, S. (2019). Persistent data storage using fstream in educational systems. Educational Technology & Society, 22(1), 56-68.

Kim, H., & Lee, J. (2021). Using classes and structs for organized data management in educational systems. Journal of Software Engineering, 15(3), 89-105.

Miller, A. (2023). Ensuring data security in college management systems. Journal of Educational Technology, 12(3), 234-250.

Smith, J., Adams, K., & Johnson, L. (2018). Implementing dynamic data management with linked lists. International Journal of Information Systems, 14(3), 45-59.
Williams, A. (2021). Performance analysis of linked lists in educational data management. Journal of Artificial Intelligence in Education, 25(2), 98-115.